

Platform Engineering & Documentation

AICB-P2T2 · Ngày 28 · Chương 6: Tổng Hợp

Giảng viên

VinUniversity

Phase 2 · Track 2 · Tuần 6

HÃY SUY NGHĨ...

*“Từng piece hoạt động riêng lẻ — nhưng khi ghép lại thành platform, thách thức mới xuất hiện ở đâu? **Milestone 3:** Hôm nay team demo end-to-end AI platform — from data ingestion to model serving với full observability.”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

1. CP2 Platform Architecture Review
2. Integration Patterns & Anti-patterns
3. End-to-End Request Flow
4. Integration Testing Strategy
5. Performance Profiling
6. Production Readiness Checklist
7. Milestone 3: Full Platform Demo
8. Labs: Platform Integration Sprint

Sau buổi học này, bạn sẽ:

1. Tích hợp toàn bộ infrastructure stack thành platform hoàn chỉnh
2. Demo end-to-end AI platform: data ingestion → model serving
3. Hoàn thành production readiness checklist cho platform
4. Present Milestone 3 demo cho instructors & peers

Architecture review (30 min) → Integration workshop (90 min) → Demo & Labs (còn lại)

Full AI infrastructure platform demo — from data ingestion to model serving với full observability

- End-to-end flow: ingest data → pipeline → model update → serving responds
- 5 smoke tests passing cho critical user journeys
- Production readiness checklist >80% complete
- Milestone 3 demo recording hoặc live presentation

- Layer 5 — Governance: RBAC + PII pipeline + encryption + compliance automation
- Layer 4 — Ops: GitHub Actions CI/CD + LangSmith LLMOps + Prometheus/Grafana
- Layer 3 — ML: MLflow experiments + DVC versioning + Feature Store (Feast)
- Layer 2 — Data: Lakehouse (Delta Lake) + Airflow + Kafka + Vector Store
- Layer 1 — Compute: Kubernetes + GPU nodes + vLLM serving + auto-scaling

Key insight: Mỗi layer đã build riêng — hôm nay ghép lại thành platform hoàn chỉnh.

Anti-pattern	Pattern	Tool
Tightly coupled components — failure cascades	Event-driven integration — Kafka decouples producers/consumers	Kafka, Redis Streams
Hardcoded config — connection strings in code	GitOps — all config in Git, deployed via ArgoCD	ArgoCD, Helm
Shared mutable state — race conditions	Immutable events + event sourcing — append-only log	Kafka topics
Manual deployment — “works on my machine”	CI/CD pipeline — automated build, test, deploy	GitHub Actions
Failure cascading across services	Bulkhead pattern — tách critical path (inference) khỏi non-critical (batch training)	K8s namespaces, resource quotas

- Producers: Data Ingestion, Airflow DAG, Model Training
- Kafka: data.raw, data.processed, model.events
- Consumers: Data Pipeline, Vector Store, Model Serving

Benefit: Producers và consumers hoàn toàn decoupled — add new consumer không impact existing pipeline.

- User Request → API Gateway → Routing Layer → Agent Orchestrator
- Parallel calls: Feature Store (<5ms), Vector Search (<50ms), LLM Inference (<500ms)
- Guardrails (PII check) → Response (total <1s)
- All calls traced: OpenTelemetry → Jaeger → LangSmith

- Input hash (privacy-safe)
- Output hash + response length
- End-to-end latency breakdown
- Token cost per component
- Model version used

- API Gateway: 5ms
- Feature Store lookup: 5ms
- Vector search: 50ms
- LLM inference: 500ms
- Guardrails check: 20ms
- **Total budget: 1000ms**

- Ensure API contracts giữa services không bị break
- Consumer-driven: consumers define expected interface
- Run in CI — block merge nếu contract violated

- Post-deploy test suite: 5 critical user journeys
- Fail fast trên production
- Run automatically after every deployment
- Test cả **golden path** (happy flow) VÀ **failure path** (error handling, timeout, retry)

- Lightweight K8s: Kind or k3d
- All services running locally
- Seeded test datasets với known expected outputs
- **Testcontainers**: spin up real Postgres, Redis, Kafka trong Docker cho integration tests — thay vì mock

- Inject latency between services
- Kill pods randomly
- Corrupt input data
- Verify graceful degradation

Tool	Target	Khi nào dùng
Jaeger (request waterfall)	E2E latency breakdown	Identify parallel vs sequential calls
cProfile / py-spy	CPU profiling	Hot spots trong preprocessing
tracemalloc	Memory allocation	Memory leaks trong long-running services
EXPLAIN ANALYZE	Database queries	Slow queries, missing indexes
tc (traffic control)	Network latency	Simulate high latency, test resilience

1. Jaeger trace → find bottleneck service **2.** cProfile/py-spy → find hot function **3.** Fix → re-profile → verify improvement

py-spy — Profile running process:

```
# Attach to running process
```

```
py-spy top --pid 12345
```

```
# Generate flamegraph
```

```
py-spy record \  
  -o profile.svg \  
  --pid 12345
```

- P50, P95, P99 latency per service
- Memory usage over time (leak?)
- CPU utilization per pod
- GPU utilization & memory
- Network I/O between services

tracemalloc — Memory tracking:

```
import tracemalloc  
tracemalloc.start()  
# ... run code ...  
snapshot = tracemalloc.take_snapshot()  
top = snapshot.statistics('lineno')  
for stat in top[:5]:
```

- Synchronous DB calls in hot path
- Missing connection pooling
- Oversized model loading
- Unoptimized vector search

- Reliability
- Observability
- Security
- Performance
- Operations

Rule: Checklist phải được **automated** — không rely vào human memory. CI pipeline check mỗi deploy.

- ☐ Health checks (liveness + readiness)
- ☐ Circuit breakers configured
- ☐ Retries with exponential backoff
- ☐ Graceful shutdown handles in-flight

- ☐ Secrets in Vault/KMS
- ☐ RBAC configured per service
- ☐ PII pipeline handling
- ☐ Security scan passing

- ☐ Logs: structured JSON
- ☐ Metrics: Prometheus exported
- ☐ Traces: OpenTelemetry configured
- ☐ Alerts: P0/P1/P2 set

- ☐ Runbooks for top 5 incidents
- ☐ Backup/restore tested
- ☐ Disaster recovery plan
- ☐ Load tested at 2x peak

End-to-end flow: ingest new data → pipeline runs → model updated → serving responds

Integration checklist: 10 integration points must work together:

1. Data ingestion → Kafka
2. Kafka → Airflow pipeline
3. Pipeline → Delta Lake / Lakehouse
4. Lakehouse → Feature Store (Feast)
5. Data → Vector Store (embeddings)
6. MLflow → Model Registry
7. Model → vLLM/SGLang serving
8. Serving → API Gateway
9. All components → Prometheus/Grafana
10. All components → LangSmith tracing

Criteria	Weight	Description
Integration Completeness	40%	All 10 integration points working, data flows end-to-end
Observability	25%	Logs, metrics, traces visible; alerts configured; SLO dashboard
Performance	20%	Latency within SLO; load tested; no memory leaks
Architecture Quality	15%	Clean separation, GitOps config, documented decisions

Config drift between environments | Missing error handling at integration points | Incomplete monitoring coverage | No rollback strategy

1. Architecture overview (2 min)
2. Live demo: happy path (5 min)
3. Live demo: error scenario (3 min)
4. Observability walkthrough (3 min)
5. Q&A from instructors/peers (2 min)

- Script the demo flow trước
- Test all happy paths AND key error scenarios
- Have fallback: pre-recorded video nếu live fails
- Show Grafana dashboard real-time
- Highlight architectural decisions & trade-offs

Mục tiêu: Full Platform Integration Sprint

Deliverable: Connect all components, write smoke tests, complete checklist, prepare demo

Thời gian: 2h

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Integration là nơi “works on my machine” meets reality — test integration surfaces trước khi production.
- 2 Production readiness checklist phải được automated — không rely vào human memory, CI pipeline check mỗi deploy.
- 3 Platform nghĩa là team khác dùng được — API contracts, documentation, SLAs quan trọng hơn internal code quality.

Hỏi & Đáp

Câu hỏi nào về platform integration, production readiness, hay Milestone 3 demo?



Cảm ơn!

AICB-P2T2 · Ngày 28

Platform Engineering & Documentation

`lms.vinuni.edu.vn` · Slide & template trên LMS